

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *ALGORITMA SORTING*
MENGGUNAKAN BAHASA *JAVA*



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 7 dengan judul “Implementasi Algoritma Sorting Menggunakan Bahasa *Java*” tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep algoritma sorting serta penerapannya dalam pemrograman *Java*. Melalui praktikum ini, penulis mempelajari bagaimana proses pengurutan data dilakukan menggunakan beberapa metode *sorting*, yaitu *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*.

Selain itu penulis juga mengimplementasikan visualisasi proses pengurutan data menggunakan *Java GUI* sehingga proses sorting dapat diamati langkah demi langkah. Dengan implementasi tersebut, penulis dapat memahami cara kerja masing-masing algoritma sorting, mulai dari proses pertukaran data, pencarian nilai minimum, hingga penyisipan elemen pada posisi yang sesuai. Praktikum ini juga membantu penulis memahami perbedaan mekanisme dan efisiensi dari setiap metode pengurutan data.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 20 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II PEMBAHASAN	7
2.1 Dasar Teori.....	7
2.1.1 <i>Bubble Sort</i>	7
2.1.2 <i>Selection Sort</i>	7
2.1.3 <i>Insertion Sort</i>	8
2.2 Langkah Kerja	9
2.2.1 Pembuatan <i>Package</i> dan <i>Class</i>	9
2.2.2 Membuat Program <i>InsertonSort_2511532002</i>	13
2.2.3 Membuat Program <i>SelectionSort_2511532002</i>	14
2.2.4 Membuat Program <i>BubleSort_2511532002 {</i>	16
2.2.5 Membuat Program <i>InsertionSortGUI_2511532002</i>	18
2.2.6 <i>Upload</i> ke GitHub	23
2.3 Hasil.....	26
2.3.1 Hasil <i>InsertonSort_2511532002</i>	26
2.3.2 Hasil <i>SelectionSort_2511532002</i>	26
2.3.3 Hasil <i>BubleSort_2511532002</i>	27
2.3.4 Hasil <i>InsertionSortGUI_2511532002</i>	27

BAB III KESIMPULAN.....	28
3.1 Kesimpulan	28
3.2 Saran	28
DAFTAR PUSTAKA.....	29
TUGAS PRAKTIKUM PEKAN 5.....	30
1. Kode Program	30
1.1 Kelas Mahasiswa_2511532002	30
1.2 Kelas MahasiswaGUI_2511532002.....	32
2. Hasil.....	37
3. Kesimpulan	39

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya kemampuan dalam mengelola data secara efisien dan terstruktur. Dalam dunia pemrograman, pengolahan data menjadi salah satu aspek penting yang harus diperhatikan agar sistem dapat bekerja dengan optimal. Salah satu bentuk pengolahan data yang paling dasar namun sangat penting adalah proses pengurutan data (*sorting*).

Sorting merupakan proses menyusun data dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Proses ini berperan penting dalam berbagai aplikasi karena dapat mempermudah pencarian, analisis, dan pengelolaan data. Dalam pemrograman Java, *sorting* dapat dilakukan dengan dua cara, yaitu menggunakan algoritma *sorting* manual maupun menggunakan fungsi bawaan seperti *Arrays.sort()*.

Pada praktikum ini, penulis mempelajari dan mengimplementasikan beberapa algoritma *sorting* dasar, yaitu *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*. Ketiga algoritma tersebut memiliki cara kerja yang berbeda dalam mengurutkan data, namun memiliki tujuan yang sama yaitu menghasilkan data yang terurut dengan benar. Selain itu, praktikum ini juga mengimplementasikan visualisasi proses *sorting* menggunakan *Java GUI (Swing)*.

Melalui praktikum ini, penulis diharapkan dapat memahami konsep dasar algoritma *sorting*, cara kerja masing-masing metode, serta mampu mengimplementasikannya dalam bahasa pemrograman *Java* sebagai dasar pengembangan aplikasi yang lebih kompleks di masa mendatang.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1.2.1 Memahami konsep dasar algoritma sorting sebagai metode pengurutan data dalam struktur pemrograman.
- 1.2.2 Mengetahui dan memahami cara kerja beberapa algoritma sorting, yaitu *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* dalam bahasa pemrograman Java.
- 1.2.3 Mengimplementasikan visualisasi proses sorting menggunakan *Java GUI (Swing)* untuk memahami proses pengurutan data secara bertahap dan interaktif.
- 1.2.4 Melatih kemampuan dalam menganalisis dalam membandingkan cara kerja dan langkah-langkah setiap algoritma *sorting*.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

- 1.3.1 Menambah pemahaman mengenai konsep dan penerapan algoritma *sorting* dalam pemrograman *Java*.
- 1.3.2 Membantu Mahasiswa memahami proses pengurutan data menggunakan berbagai metode sorting seperti *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*. Serta visualisasi algoritma menggunakan *Java GUI* sehingga proses sorting dapat diamati secara langsung.
- 1.3.3 Melatih kemampuan logika pemrograman dalam mengelola dan mengurutkan data secara sistematis.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting adalah proses pengurutan data berdasarkan aturan tertentu, biasanya dalam urutan menaik (*ascending*) atau menurun (*descending*). Dalam dunia pemrograman, *sorting* digunakan untuk mempermudah pencarian, pengolahan, dan analisis data.

Algoritma *sorting* merupakan langkah-langkah sistematis yang digunakan untuk mengurutkan elemen data dalam struktur seperti *array* atau *list*. Setiap algoritma memiliki cara kerja yang berbeda, namun tujuan akhirnya sama yaitu menghasilkan data yang terurut.

2.1.1 *Bubble Sort*

Bubble Sort adalah algoritma *sorting* yang bekerja dengan cara membandingkan dua elemen yang bersebelahan, kemudian menukarnya jika urutannya salah.

Proses ini dilakukan secara berulang hingga seluruh data berada dalam keadaan terurut.

Karakteristik *Bubble Sort*:

1. Sederhana dan mudah dipahami
2. Kurang efisien untuk data besar
3. Kompleksitas waktu: $O(n^2)$

2.1.2 *Selection Sort*

Selection Sort adalah algoritma pengurutan yang bekerja dengan cara mencari elemen terkecil (atau terbesar) dari data, kemudian menukarnya dengan elemen di posisi awal. Langkah ini dilakukan secara berulang hingga seluruh data terurut.

Karakteristik *Selection Sort*:

1. Jumlah pertukaran data lebih sedikit dibanding Bubble Sort
2. Tetap memiliki kompleksitas $O(n^2)$
3. Cocok untuk pembelajaran dasar *sorting*

2.1.3 Insertion Sort

Insertion Sort bekerja dengan cara menyisipkan setiap elemen ke posisi yang tepat dalam bagian data yang sudah terurut. Algoritma ini mirip dengan cara mengurutkan kartu di tangan.

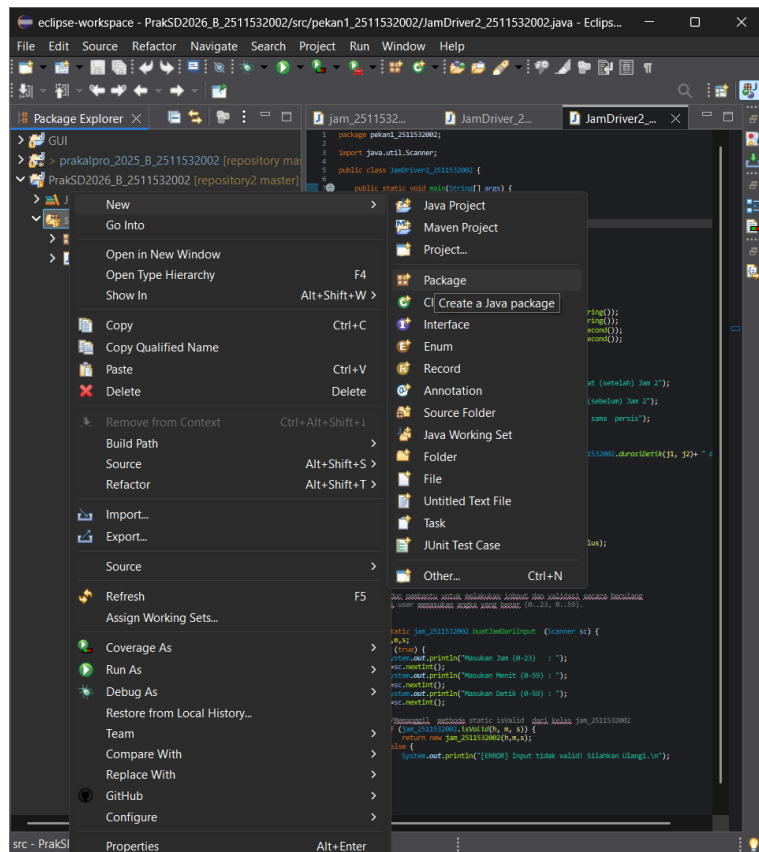
Karakteristik *Insertion Sort*:

1. Efektif untuk data kecil atau hampir terurut
2. Lebih efisien dibanding Bubble Sort dalam beberapa kasus
3. Kompleksitas rata-rata $O(n^2)$

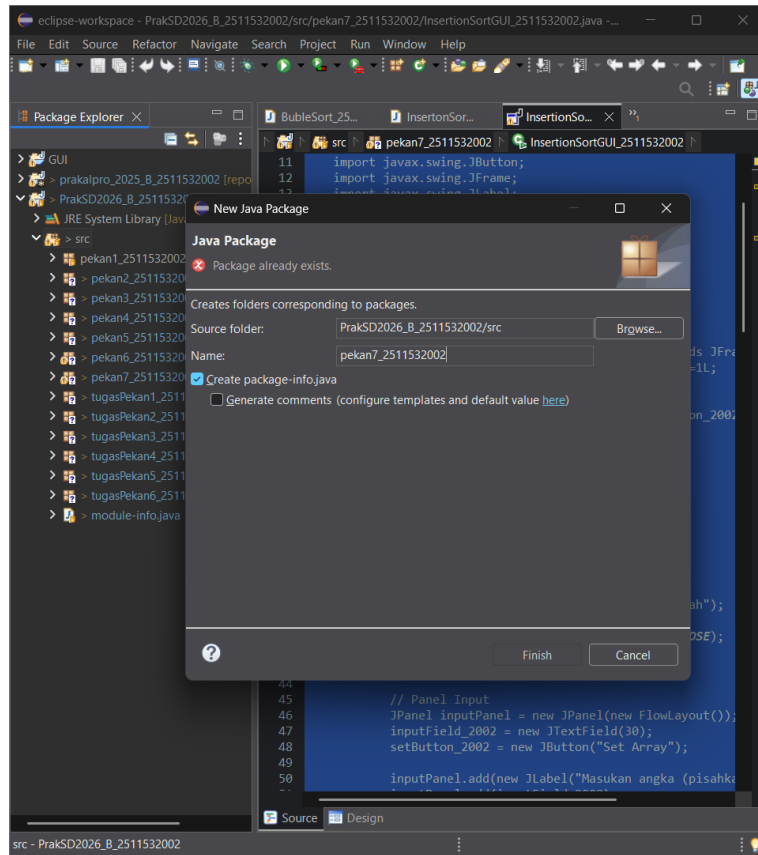
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

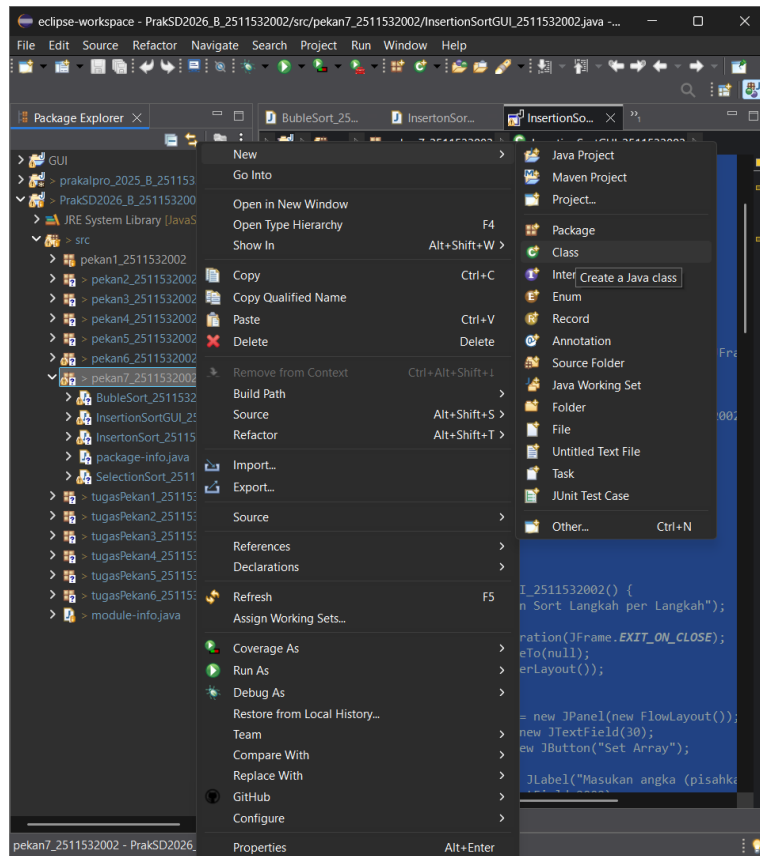
- 1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* “PrakSD2026_B_2511532002” lalu pilih “New” dan klik “Package”.



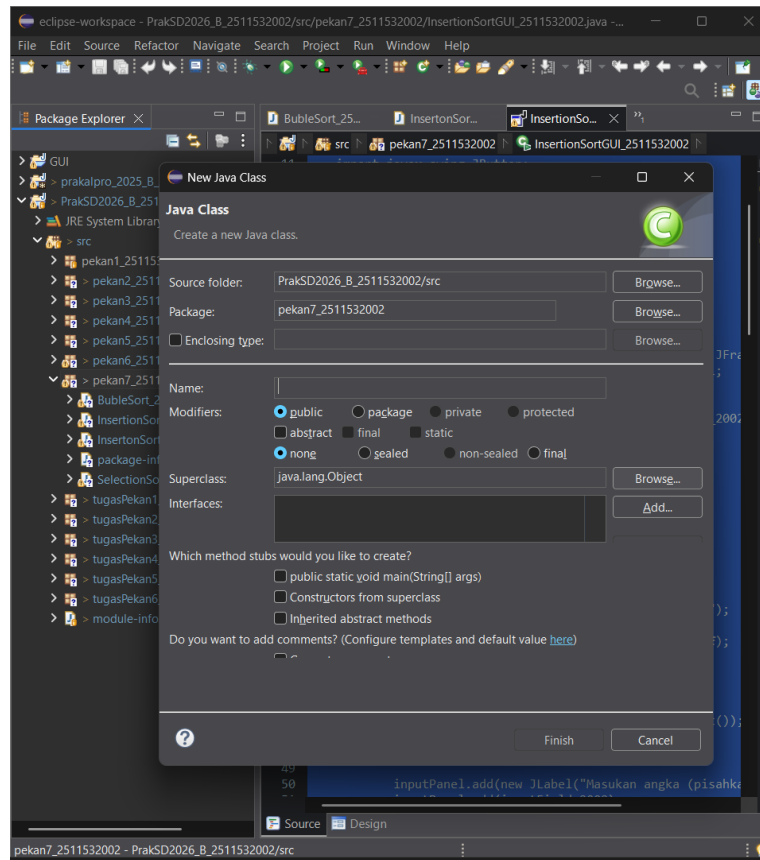
- 2) Setelah itu akan muncul *pop up* “Java Package”, buat nama *package* dengan ketentuan tanpa *capslock*, *space*, dan karakter khusus lainnya. Lalu klik “*finish*”.



- 3) Selajutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih “New” dan klik “Class”.



- 4) Setelah itu pada menu “*Java Class*” buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *InsertonSort_2511532002*

```
package pekan7_2511532002;

public class InsertonSort_2511532002 {
    public static void InsertonSort_2511532002 (int[] arr) {
        int n_2002 = arr.length;
        for (int i_2002 = 1; i_2002 < n_2002; i_2002++) {
            int key_2002 = arr[i_2002];
            int j_2002 = i_2002 - 1;
            while (j_2002 >= 0 && arr[j_2002] > key_2002) {
                arr[j_2002+1] = arr[j_2002];
                j_2002--;
            }
            arr[j_2002+1] = key_2002;
        }
    }
    public static void main(String[] args) {
        int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
        int n_2002 = arr_2002.length;
        System.out.println("array yang belum terurut:\n");
        for (int i_2002 = 0; i_2002 < n_2002; i_2002++)
            System.out.print(arr_2002[i_2002] + " ");
        System.out.println("");
        InsertonSort_2511532002(arr_2002);
        System.out.println("array yang terurut:\n");
        for (int i_2002 = 0; i_2002 < n_2002; i_2002++)
            System.out.print(arr_2002[i_2002] + " ");
        System.out.println("");
    }
}
```

Setelah membuat *class InsertonSort_2511532002*, maka mahasiswa dapat menulis *syntax* nya.

1. Membuat *class Insertion Sort*

```
public class InsertonSort_2511532002 {
```

Syntax di atas digunakan untuk membuat class bernama *InsertonSort_2511532002* sebagai tempat penulisan program algoritma *Insertion Sort*.

2. Membuat *method Insertion Sort*

```
public static void InsertonSort_2511532002 (int[] arr) {
    int n_2002 = arr.length;
    for (int i_2002 = 1; i_2002 < n_2002; i_2002++) {
        int key_2002 = arr[i_2002];
        int j_2002 = i_2002 - 1;
        while (j_2002 >= 0 &&
arr[j_2002] > key_2002) {
            arr[j_2002+1] = arr[j_2002];
            j_2002--;
        }
        arr[j_2002+1] = key_2002;
    }
}
```

Membuat *methode Inertion Sort* untuk melakukan proses pengurutan data menggunakan algoritma Insertion Sort dengan tipe data *array integer* dengan perulangan *for* untuk memproses elemen array satu persatu dan dengan *while* untuk membandingkan dan menggeser elemen.

3. Membuat *method main*

```
public static void main(String[] args) {
    int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
    int n_2002 = arr_2002.length;
    System.out.println("array yang belum terurut:\n");
    for (int i_2002=0; i_2002<n_2002; i_2002++)
        System.out.print(arr_2002[i_2002]+" ");
    System.out.println("");
    InsertonSort_2511532002(arr_2002);
    System.out.println("array yang terurut:\n");
    for (int i_2002=0; i_2002<n_2002; i_2002++)
        System.out.print(arr_2002[i_2002]+" ");
    System.out.println("");
}
```

Method main digunakan sebagai *method* utama untuk menjalankan program. Dengan menampilkan *array* sebelum terurut dan setelah terurut

2.2.3 Membuat Program *SelectionSort_2511532002*

```
package pekan7_2511532002;

public class SelectionSort_2511532002 {
    public static void SelectionSort_2511532002(int[] arr_2002) {
        int n_2002 = arr_2002.length;
        for (int i_2002 = 0; i_2002 < n_2002; i_2002++) {
            int minIndex_2002 = 1;
            for (int j = i_2002 + 1; j < n_2002; j++) {
                if (arr_2002[j] < arr_2002[minIndex_2002]) {
                    minIndex_2002 = j;
                }
            }
            int temp_2002 = arr_2002[i_2002];
            arr_2002[i_2002] = arr_2002[minIndex_2002];
            arr_2002[minIndex_2002] = temp_2002;
        }
    }

    public static void main(String[] args) {
        int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
        int n_2002 = arr_2002.length;
        System.out.println("array yang belum terurut:\n");
        for (int i_2002i = 0; i_2002i < n_2002; i_2002i++)
            System.out.print(arr_2002[i_2002i] + " ");
        System.out.println("");
        SelectionSort_2511532002(arr_2002);
        System.out.println("array yang terurut:\n");
        for (int i_2002 = 0; i_2002 < n_2002; i_2002++)
            System.out.print(arr_2002[i_2002] + " ");
        System.out.println("");
    }
}
```

1. Membuat *class Selection Sort*

```
public class SelectionSort_2511532002 {
```

Syntax di atas digunakan untuk membuat class bernama *SelectionSort_2511532002* sebagai tempat penulisan program algoritma *Selection Sort*.

2. Membuat *method Selection Sort*

```
public static void SelectionSort_2511532002(int[] arr_2002) {
    int n_2002 = arr_2002.length;
    for (int i_2002 = 0; i_2002 < n_2002; i_2002++) {
        int minIndex_2002 = 1;
        for (int j = i_2002 + 1; j < n_2002; j++) {
            if (arr_2002[j] <
arr_2002[minIndex_2002]) {
                minIndex_2002 = j;
            }
        }
        int temp_2002 = arr_2002[i_2002];

        arr_2002[i_2002] = arr_2002[minIndex_2002];
        arr_2002[minIndex_2002] = temp_2002;
    }
}
```

Membuat *method SelectionSort_2511532002* untuk melakukan proses pengurutan data menggunakan algoritma *Selection Sort* dengan tipe data *array integer*. Proses sorting dilakukan menggunakan perulangan *for* untuk mencari nilai terkecil pada *array*, kemudian menukarkan posisi data menggunakan variabel sementara (*temp_2002*).

3. Membuat *method main*.

```
public static void main(String[] args) {
    int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
    int n_2002 = arr_2002.length;
    System.out.println("array yang belum terurut:\n");
    for (int i_2002 = 0; i_2002 < n_2002; i_2002++)
        System.out.print(arr_2002[i_2002] + " ");
    System.out.println("");
    SelectionSort_2511532002(arr_2002);
    System.out.println("array yang terurut:\n");
    for (int i_2002 = 0; i_2002 < n_2002; i_2002++)
        System.out.print(arr_2002[i_2002] + " ");
}
```

```

        System.out.println("");
    }

```

Method main digunakan sebagai *method* utama untuk menjalankan program dengan menampilkan *array* sebelum terurut dan setelah terurut menggunakan algoritma *Selection Sort*.

2.2.4 Membuat Program *BubleSort_2511532002* {

```

package pekan7_2511532002;

public class BubleSort_2511532002 {
    public static void BubleSort_2511532002(int[] arr) {
        int n_2002=arr.length;
        for (int i_2002=0; i_2002 <n_2002; i_2002++) {
            for (int j_2002=0; j_2002<n_2002 - i_2002-1;
j_2002++) {
                if (arr[j_2002]>arr[j_2002+1]) {
                    int temp_2002=arr[j_2002];
                    arr[j_2002]=arr[j_2002+1];
                    arr[j_2002+1]=temp_2002;
                    System.out.println("data:
"+arr[j_2002]+" "+arr[j_2002+1]);
                }
            }
        }

        public static void main(String[] args) {
            int arr_2002[]= {23, 78, 45, 8, 32, 56, 1};
            int n_2002 = arr_2002.length;
            System.out.println("array yang belum terurut:\n");
            for (int i_2002=0; i_2002<n_2002;i_2002++)
                System.out.print(arr_2002[i_2002]+" ");
            System.out.println("");
            BubleSort_2511532002(arr_2002);
            System.out.println("array yang terurut:\n");
            for (int i_2002=0; i_2002<n_2002; i_2002++)
                System.out.print(arr_2002[i_2002]+" ");
            System.out.println("");
        }
    }
}

```

1. Membuat *class Bubble Sort*

```
public class BubleSort_2511532002 {
```

Syntax di atas digunakan untuk membuat *class* bernama *BubleSort_2511532002* sebagai tempat penulisan program algoritma *Bubble Sort*.

2. Membuat *method Bubble Sort*

```

public static void BubleSort_2511532002(int[] arr) {
    int n_2002=arr.length;
    for (int i_2002=0; i_2002 <n_2002; i_2002++) {

```



```

        for (int j_2002=0; j_2002<n_2002 - i_2002-1;
j_2002++) {
            if (arr[j_2002]>arr[j_2002+1]) {
                int temp_2002=arr[j_2002];
                arr[j_2002]=arr[j_2002+1];
                arr[j_2002+1]=temp_2002;
                System.out.println("data:
"+arr[j_2002]+" "+arr[j_2002+1]);
            }
        }
    }
}

```

Membuat method *BubleSort_2511532002* untuk melakukan proses pengurutan data menggunakan algoritma *Bubble Sort* dengan tipe data *array integer*. Proses *sorting* dilakukan menggunakan perulangan *for* untuk membandingkan dua elemen yang bersebelahan, kemudian menukarkan posisi data jika urutannya salah menggunakan variabel sementara (*temp_2002*).

3. Membuat method *main*

```

public static void main(String[] args) {
    int arr_2002[]={23, 78, 45, 8, 32, 56, 1};
    int n_2002 = arr_2002.length;
    System.out.println("array yang belum terurut:\n");
    for (int i_2002=0; i_2002<n_2002; i_2002++)
        System.out.print(arr_2002[i_2002]+" ");
    System.out.println("");
    BubleSort_2511532002(arr_2002);
    System.out.println("array yang terurut:\n");
    for (int i_2002=0; i_2002<n_2002; i_2002++)
        System.out.print(arr_2002[i_2002]+" ");
    System.out.println("");
}
}

```

Method *main* digunakan sebagai method utama untuk menjalankan program dengan menampilkan array sebelum terurut dan setelah terurut menggunakan algoritma *Bubble Sort*.

2.2.5 Membuat Program *InsertionSortGUI_2511532002*

```

package pekan7_2511532002;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511532002 extends JFrame {
    private static final long serialVersionUID=1L;
    private int[] array_2002;
    private JLabel[] labelArray_2002;
    private JButton stepButton_2002,
resetButton_2002, setButton_2002;
    private JTextField inputField_2002;
    private JPanel panelArray_2002;
    private JTextArea stepArea_2002;

    private int i= 1, j;
    private boolean sorting = false;
    private int stepCount=1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532002() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel Input
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_2002 = new JTextField(30);
        setButton_2002 = new JButton("Set Array");

        inputPanel.add(new JLabel("Masukan angka
(pisahkan dengan koma): "));
        inputPanel.add(inputField_2002);
        inputPanel.add(setButton_2002);

        // Panel array visual
        panelArray_2002 = new JPanel();
        panelArray_2002.setLayout(new FlowLayout());

        // Panel Kontrol
        JPanel controlPanel = new JPanel();
        stepButton_2002 = new JButton("Langkah
Selanjutnya");
        resetButton_2002 = new JButton("Reset");
        stepButton_2002.setEnabled(false);

        controlPanel.add(stepButton_2002);
        controlPanel.add(resetButton_2002);
    }
}

```

```

        // Area teks untuk log langkah-langkah
        stepArea_2002 = new JTextArea(8, 60);
        stepArea_2002.setEditable(false);
        stepArea_2002.setFont(new Font("Monospaced",
Font.PLAIN, 14));

        JScrollPane scrollPane = new
JScrollPane(stepArea_2002);

        //Tambahan Panel ke frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_2002, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        //Event set array
        setButton_2002.addActionListener(e->
setArrayFromInput());

        //Event langkah selanjutnya
        stepButton_2002.addActionListener(e->
performStep());

        //Event reset
        resetButton_2002.addActionListener(e-> reset ());
    }

    private void setArrayFromInput() {
        String text = inputField_2002.getText().trim();
        if (text.isEmpty()) return;
        String [] parts=text.split(",");
        array_2002=new int[parts.length];
        try {
            for (int k=0; k< parts.length; k++) {
                array_2002[k]
=Integer.parseInt(parts[k].trim());
            }catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(this,
"Masukan hanya angka yang dipisahkan"+"dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
                return;
            }

            i=1;
            stepCount=1;
            sorting=true;
            stepButton_2002.setEnabled(true);
            stepArea_2002.setText("");
            panelArray_2002.removeAll();
            labelArray_2002=new JLabel[array_2002.length];
            for (int k=0; k < array_2002.length; k++) {
                labelArray_2002[k] = new
JLabel(String.valueOf(array_2002[k]));
                labelArray_2002[k].setFont(new Font
("Arial", Font.BOLD, 24));

                labelArray_2002[k].setBorder(BorderFactory.createLineBorder(Color
.BLACK));

                labelArray_2002[k].setPreferredSize(new
Dimension(50, 50));

                labelArray_2002[k].setHorizontalAlignment(SwingConstants.CENTER);
                panelArray_2002.add(labelArray_2002[k]);
            }
            panelArray_2002.revalidate();
            panelArray_2002.repaint();
        }
        private void performStep() {
            if (i<array_2002.length&& sorting) {
                int key=array_2002[i];
                j=i-1;

```

```

        StringBuilder stepLog = new
StringBuilder();

        stepLog.append("Langkah").append(stepCount).append(": Memasukan
").append(key).append("\n");

        while (j >= 0 && array_2002[j] > key) {
            array_2002[j + 1] = array_2002[j];
            j--;
        }

        array_2002[j + 1] = key;

        updateLabels();
        stepLog.append("Hasil:
").append(arrayToString(array_2002)).append("\n\n");
        stepArea_2002.append(stepLog.toString());

        i++;
        stepCount++;

        if (i == array_2002.length) {
            sorting = false;
            stepButton_2002.setEnabled(false);
            JOptionPane.showMessageDialog(this,
"Sorting selesai!");
        }
    }

    private void updateLabels() {
        for (int k=0; k<array_2002.length; k++) {
            labelArray_2002[k].setText(String.valueOf(array_2002[k]));
        }
    }

    private void reset() {
        inputField_2002.setText("");
        panelArray_2002.removeAll();
        panelArray_2002.revalidate();
        panelArray_2002.repaint();
        stepArea_2002.setText("");
        stepButton_2002.setEnabled(false);
        sorting = false;
        i = 1;
        stepCount = 1;
    }

    private String arrayToString(int[] arr) {
        StringBuilder sb = new StringBuilder();

        for (int k = 0; k < arr.length; k++) {
            sb.append(arr[k]);

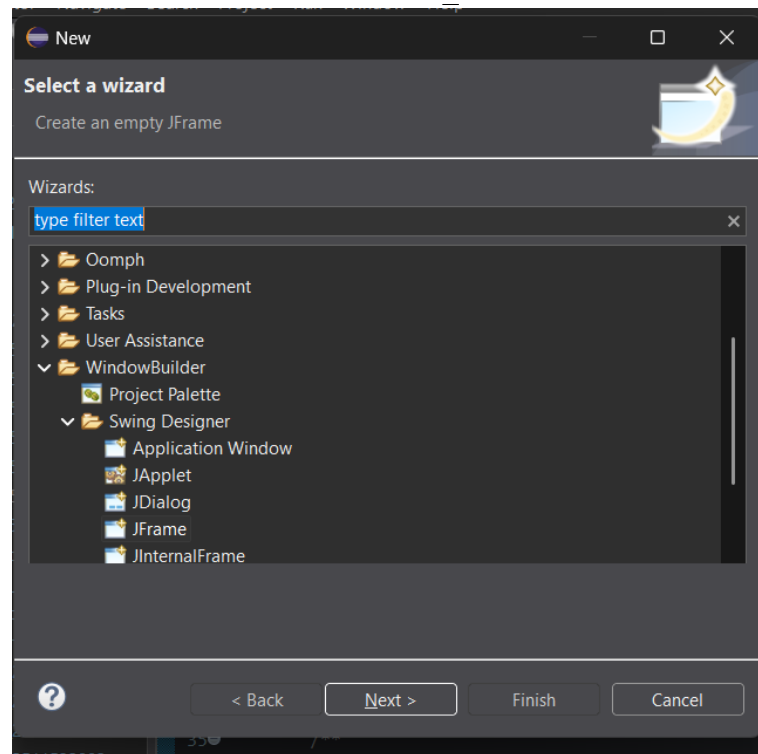
            if (k < arr.length - 1)
                sb.append(", ");
        }

        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511532002 gui =
            new InsertionSortGUI_2511532002();
            gui.setVisible(true);
        });
    }
}

```

1. Membuat kelas *InsertionSortGUI_2511532002*



Klik kanan pada *package*, lalu pilih “New” lalu pilih “Other”. Selanjutnya pilih “JFrame”.

2. Menulis syntax program, untuk menulis *syntax* buka tab “Source” maka akan terbuka tab *syntax* program *InsertionSortGUI_2511532002*.

3. Membuat *class GUI Insertion Sort*
`public class InsertionSortGUI_2511532002 extends JFrame {`

Syntax di atas digunakan untuk membuat class GUI bernama *InsertionSortGUI_2511532002* yang mewarisi *JFrame* sebagai tampilan utama program visualisasi *Insertion Sort*.

4. Membuat *atribut* dan komponen *GUI*
`private int[] array_2002;
private JLabel[] labelArray_2002;
private JButton stepButton_2002, resetButton_2002,
setButton_2002;
private JTextField inputField_2002;
private JPanel panelArray_2002;
private JTextArea stepArea_2002;`

Syntax di atas digunakan untuk membuat atribut program dan komponen *GUI* seperti tombol, panel, *text field*, label, dan area teks yang digunakan untuk menampilkan visualisasi proses *sorting*.

5. Membuat *constructor GUI*

```
public InsertionSortGUI_2511532002() {
```

Constructor digunakan untuk mengatur tampilan *GUI* seperti ukuran *frame*, *layout*, panel *input*, panel visualisasi array, tombol kontrol, dan area log langkah-langkah *sorting*.

6. Membuat *panel input*

```
JPanel inputPanel = new JPanel(new FlowLayout());
inputField_2002 = new JTextField(30);
setButton_2002 = new JButton("Set Array");
```

Syntax di atas digunakan untuk membuat panel input yang berfungsi menerima data array dari pengguna melalui *JTextField* dan tombol *Set Array*.

7. Membuat *method setArrayFromInput*

```
private void setArrayFromInput() {
```

Method ini digunakan untuk mengambil *input array* dari pengguna, mengubah data menjadi *array integer*, serta menampilkan array ke dalam *GUI*.

8. Membuat *method performStep*

```
private void performStep() {
```

Method ini digunakan untuk menjalankan proses *Insertion Sort* langkah demi langkah dengan menampilkan perubahan array pada setiap proses *sorting*.

9. Membuat *method updateLabels*

```
private void updateLabels() {
```

Method ini digunakan untuk memperbarui tampilan label array pada GUI setelah proses sorting dilakukan.

10. Membuat *method reset*

```
private void reset() {
```

Method ini digunakan untuk menghapus seluruh data *input*, tampilan *array*, dan log sorting sehingga program kembali ke kondisi awal.

11. Membuat *method arrayToString*

```
private String arrayToString(int[] arr) {
```

Method ini digunakan untuk mengubah isi array menjadi bentuk string agar dapat ditampilkan pada area log langkah sorting.

12. Membuat *method main*

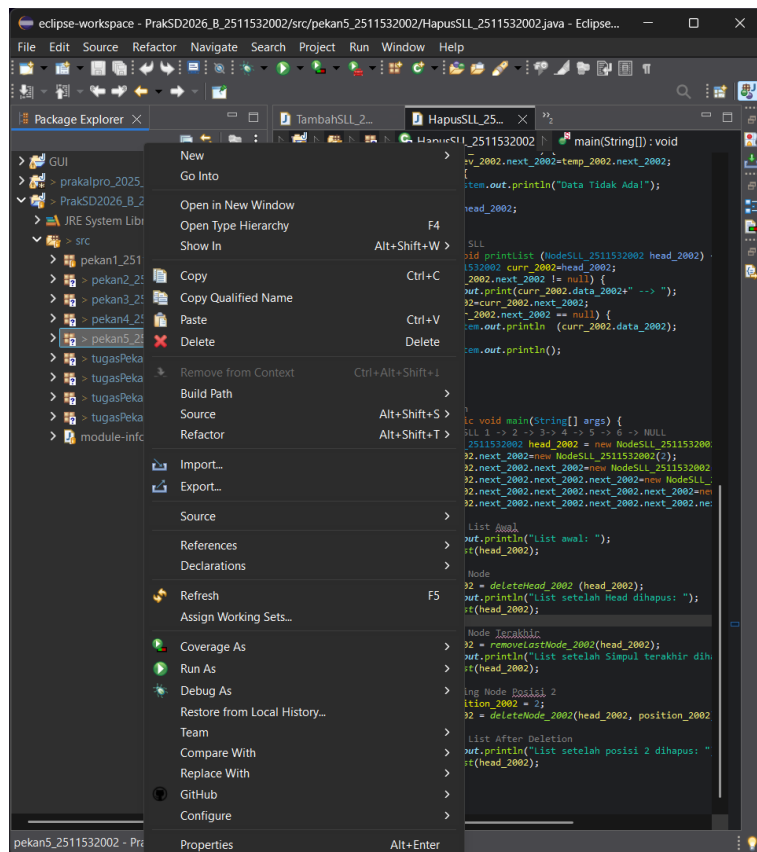
```
public static void main(String[] args) {
```

Method main digunakan sebagai method utama untuk menjalankan program *GUI Insertion Sort* menggunakan *SwingUtilities.invokeLater()* sehingga tampilan *GUI* dapat muncul dan dijalankan dengan baik.

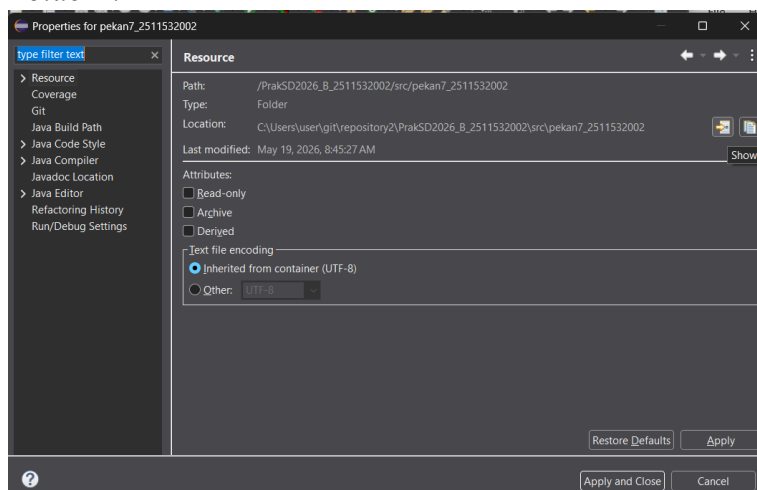
2.2.6 Upload ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclipse ke *repository* GitHub. Selain dengan cara “*Team*” atau “*Commit and Push*”.

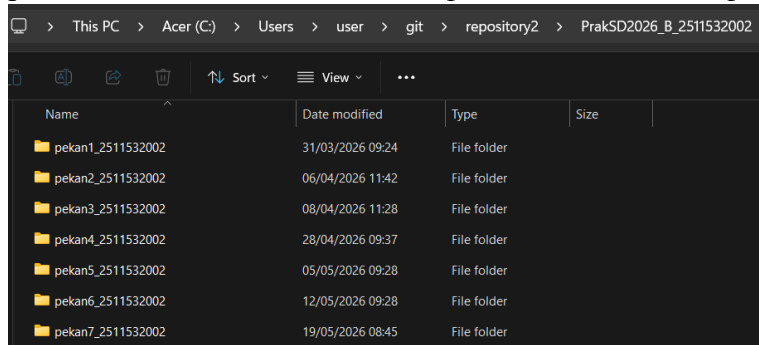
Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package*, lalu pilih *properties*.



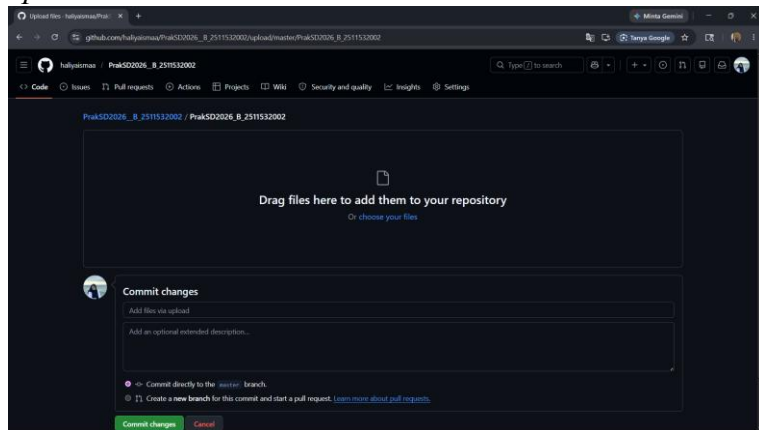
2. Maka akan muncul *pop up* menu “Properteis for pekan7_2511532002”. Lalu klik icon “Show in System Folder”.



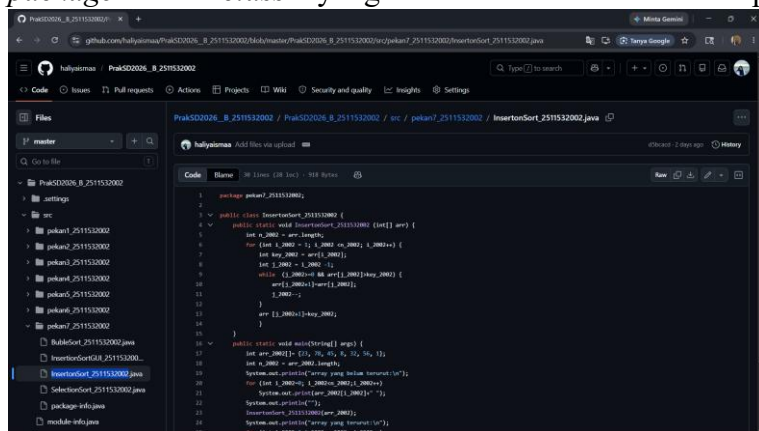
3. Maka akan menunjukan lokasi file “*pekan7_2511532002*” pada *File Explorer* computer.



4. Setelah itu *drag and drop* file “*pekan6_2511532002*” ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload*.

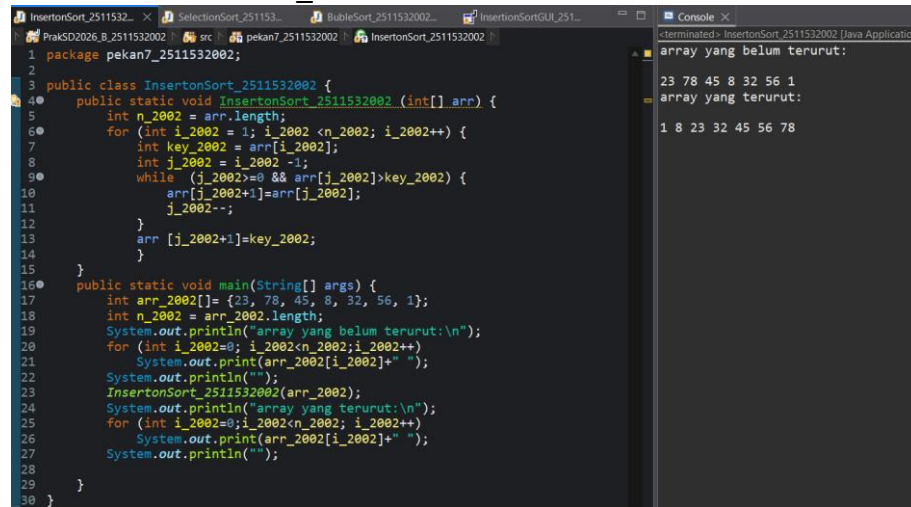


5. Maka dapat dilihat pada *repository GitHub* bahwa file “*pekan7_2511532002*” telah terunggah via *Upload*. Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil *InsertionSort_2511532002*



```

1 package pekan7_2511532002;
2
3 public class InsertionSort_2511532002 {
4     public static void InsertionSort_2511532002 (int[] arr) {
5         int n_2002 = arr.length;
6         for (int i_2002 = 1; i_2002 < n_2002; i_2002++) {
7             int key_2002 = arr[i_2002];
8             int j_2002 = i_2002 - 1;
9             while (j_2002 >= 0 && arr[j_2002] > key_2002) {
10                 arr[j_2002+1] = arr[j_2002];
11                 j_2002--;
12             }
13             arr[j_2002+1] = key_2002;
14         }
15     }
16     public static void main(String[] args) {
17         int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
18         int n_2002 = arr_2002.length;
19         System.out.println("array yang belum terurut:\n");
20         for (int i_2002=0; i_2002<n_2002; i_2002++)
21             System.out.print(arr_2002[i_2002]+" ");
22         System.out.println("");
23         InsertionSort_2511532002(arr_2002);
24         System.out.println("array yang terurut:\n");
25         for (int i_2002=0; i_2002<n_2002; i_2002++)
26             System.out.print(arr_2002[i_2002]+" ");
27         System.out.println("");
28     }
29 }
30 }

```

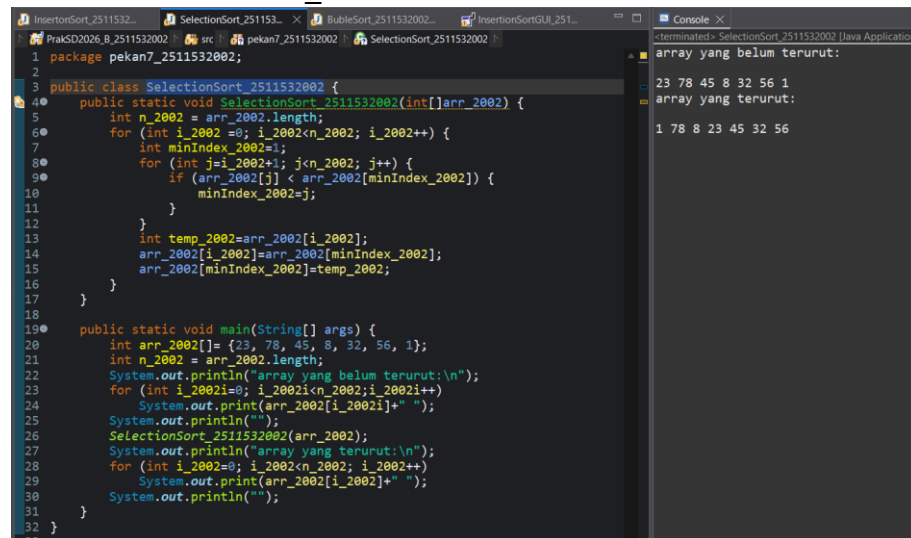
Console Output:

```

array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

2.3.2 Hasil *SelectionSort_2511532002*



```

1 package pekan7_2511532002;
2
3 public class SelectionSort_2511532002 {
4     public static void SelectionSort_2511532002(int[] arr_2002) {
5         int n_2002 = arr_2002.length;
6         for (int i_2002 = 0; i_2002 < n_2002; i_2002++) {
7             int minIndex_2002 = i_2002;
8             for (int j = i_2002+1; j < n_2002; j++) {
9                 if (arr_2002[j] < arr_2002[minIndex_2002]) {
10                     minIndex_2002 = j;
11                 }
12             }
13             int temp_2002 = arr_2002[i_2002];
14             arr_2002[i_2002] = arr_2002[minIndex_2002];
15             arr_2002[minIndex_2002] = temp_2002;
16         }
17     }
18     public static void main(String[] args) {
19         int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
20         int n_2002 = arr_2002.length;
21         System.out.println("array yang belum terurut:\n");
22         for (int i_2002=0; i_2002<n_2002; i_2002++)
23             System.out.print(arr_2002[i_2002]+" ");
24         System.out.println("");
25         SelectionSort_2511532002(arr_2002);
26         System.out.println("array yang terurut:\n");
27         for (int i_2002=0; i_2002<n_2002; i_2002++)
28             System.out.print(arr_2002[i_2002]+" ");
29         System.out.println("");
30     }
31 }
32 }
33 }

```

Console Output:

```

array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 78 8 23 45 32 56

```

2.3.3 Hasil BubleSort_2511532002

```

1 package pekan7_2511532002;
2
3 public class BubleSort_2511532002 {
4     public static void BubleSort_2511532002(int[] arr) {
5         int n_2002=arr.length;
6         for (int i_2002=0; i_2002 < n_2002; i_2002++) {
7             for (int j_2002=0; j_2002 < n_2002 - i_2002; j_2002++) {
8                 if (arr[j_2002] > arr[j_2002+1]) {
9                     int temp_2002=arr[j_2002];
10                    arr[j_2002]=arr[j_2002+1];
11                    arr[j_2002+1]=temp_2002;
12                    System.out.println("data: "+arr[j_2002]+" "+arr[j_2002+1]);
13                }
14            }
15        }
16    }
17
18    public static void main(String[] args) {
19        int arr_2002[] = {23, 78, 45, 8, 32, 56, 1};
20        int n_2002 = arr_2002.length;
21        System.out.println("array yang belum terurut:\n");
22        for (int i_2002=0; i_2002 < n_2002; i_2002++) {
23            System.out.print(arr_2002[i_2002]+" ");
24        }
25        BubleSort_2511532002(arr_2002);
26        System.out.println("array yang terurut:\n");
27        for (int i_2002=0; i_2002 < n_2002; i_2002++) {
28            System.out.print(arr_2002[i_2002]+" ");
29        }
30    }
31 }

```

Console Output:

```

array yang belum terurut:
23 78 45 8 32 56 1
data: 45 78
data: 8 78
data: 32 78
data: 56 78
data: 1 78
data: 8 45
data: 32 45
data: 1 56
data: 8 23
data: 1 45
data: 1 32
data: 1 23
data: 1 8
array yang terurut:
1 8 23 32 45 56 78

```

2.3.4 Hasil InsertionSortGUI_2511532002

```

19 import javax.swing.SwingConstants;
20 import javax.swing.SwingUtilities;
21 import javax.swing.border.EmptyBorder;
22
23 public class InsertionSortGUI_2511532002 extends JFrame {
24     private static final long serialVersionUID = 1L;
25     private int[] array_2002;
26     private JLabel labelArray_2002;
27     private JButton stepButton_2002, resetButton_2002, setButton_2002;
28     private JTextField inputField_2002;
29     private JPanel panelArray_2002;
30     private JTextArea stepArea_2002;
31
32     private int i = 1, j;
33     private boolean sorting = false;
34     private int stepCount = 1;
35
36     /**
37      * Create the frame.
38      */
39     public InsertionSortGUI_2511532002() {
40         setTitle("Insertion Sort Langkah per Langkah");
41         setSize(400, 400);
42         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43         setLocationRelativeTo(null);
44         setLayout(new BorderLayout());
45
46         // Panel Input
47         JPanel inputPanel = new JPanel(new FlowLayout());
48         inputField_2002 = new JTextField(80);
49         setButton_2002 = new JButton("Set Array");
50         inputPanel.add(new JLabel("Masukan angka (pisahkan dengan koma):"));
51         inputPanel.add(inputField_2002);
52         inputPanel.add(setButton_2002);
53
54         // Panel array visual
55         panelArray_2002 = new JPanel();

```

GUI Output:

Masukan angka (pisahkan dengan koma): 22,4,6

Langkah: Memasukan 4
Hasil: 4, 22, 6

Langkah: Memasukan 6
Hasil: 4, 6, 22

Buttons: Langkah Selanjutnya, Reset

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* merupakan metode yang digunakan untuk mengurutkan data ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Dalam praktikum ini, algoritma *sorting* berhasil diimplementasikan menggunakan bahasa pemrograman *Java* dengan beberapa metode, yaitu *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*.

Setiap algoritma memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. *Bubble Sort* dengan membandingkan elemen yang bersebelahan, *Selection Sort* dengan mencari nilai minimum, sedangkan *Insertion Sort* dengan menyisipkan elemen pada posisi yang sesuai.

Selain itu, praktikum ini juga berhasil mengimplementasikan visualisasi proses *Insertion Sort* menggunakan *Java GUI (Swing)*, sehingga proses pengurutan data dapat diamati secara bertahap dan lebih mudah dipahami. Melalui praktikum ini, penulis dapat memahami konsep dasar *sorting*, cara kerja masing-masing algoritma, serta penerapan visualisasi algoritma dalam pemrograman *Java*.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami logika algoritma *sorting* serta langkah-langkah proses pengurutan data pada setiap metode..

DAFTAR PUSTAKA

- [1] *GeeksforGeeks*, “Sorting in Java,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/java/sorting-in-java/> Diakses pada: 20 Mei 2026.
- [2] *W3Schools*, “Java Arrays Sort Method,” 2025. [Online]. Available: https://www.w3schools.com/java/ref_arrays_sort.asp Diakses pada: 20 Mei 2026.

TUGAS PRAKTIKUM PEKAN 5

1. Kode Program

1.1 Kelas *Mahasiswa_2511532002*

```
package tugasPekan7_2511532002;

public class Mahasiswa_2511532002 {
    private String nama_2002;
    private String nim_2002;
    private String prodi_2002;

    //Konstruktor
    public Mahasiswa_2511532002 (String nama_2002, String nim_2002, String
prodi_2002) {
        this.nama_2002=nama_2002;
        this.nim_2002=nim_2002;
        this.prodi_2002=prodi_2002;    }

    //getter
    public String getNama_2002() {
        return nama_2002;
    }
    public String getNim_2002() {
        return nim_2002;
    }
    public String getProdi() {
        return prodi_2002;
    }

    //setter
    public void setNama_2002(String nama_2002) {
        this.nama_2002=nama_2002;
    }
    public void setNim_2002(String nim_2002) {
        this.nim_2002=nim_2002;
    }
    public void setProdi_2002(String prodi_2002) {
        this.prodi_2002=prodi_2002;
    }

    @Override
    public String toString() {
        return nama_2002+" - "+nim_2002+" - "+prodi_2002;
    }
}
```

Penjelasan:

1) Membuat *class*

```
public class Mahasiswa_2511532002 {
```

Syntax di atas digunakan untuk membuat *class* bernama *Mahasiswa_2511532002* sebagai *ADT (Abstract Data Type)* untuk menyimpan data mahasiswa.

2) Membuat atribut

```
private String nama_2002;
private String nim_2002;
private String prodi_2002;
```

Syntax di atas digunakan untuk membuat atribut mahasiswa yang terdiri dari nama, NIM, dan program studi dengan tipe data *String*.

3) Membuat konstruktor

```
public Mahasiswa_2511532002 (String nama_2002, String
nim_2002, String prodi_2002) {
    this.nama_2002=nama_2002;
    this.nim_2002=nim_2002;
    this.prodi_2002=prodi_2002;}

```

Konstruktor digunakan untuk menginisialisasi data mahasiswa saat object dibuat, yaitu data nama, NIM, dan program studi.

4) Membuat *Getter*

```
public String getNama_2002() {
    return nama_2002;
}
public String getNim_2002() {
    return nim_2002;
}
public String getProdi() {
    return prodi_2002;
}

```

Getter digunakan untuk mengambil atau menampilkan nilai atribut mahasiswa seperti nama, NIM, dan program studi.

5) Membuat *Setter*

```
public void setNama_2002(String nama_2002) {
    this.nama_2002=nama_2002;
}
public void setNim_2002(String nim_2002) {
    this.nim_2002=nim_2002;
}

```

```

    public void setProdi_2002(String prodi_2002) {
        this.prodi_2002=prodi_2002;
    }

```

Setter digunakan untuk mengubah atau mengatur nilai atribut mahasiswa seperti nama, NIM, dan program studi.

6) Membuat *methode toString*

```

    @Override
    public String toString() {
        return nama_2002+" - "+nim_2002+" - "+prodi_2002;
    }

```

Method toString() digunakan untuk menampilkan data mahasiswa dalam bentuk *string* dengan format nama, NIM, dan program studi.

1.2 Kelas *MahasiswaGUI_2511532002*

```

package tugasPekan7_2511532002;

import java.awt.BorderLayout;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.util.ArrayList;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSplitPane;
import javax.swing.JTable;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;

public class MahasiswaGUI_2511532002 extends JFrame {

    class Mahasiswa_2002 {
        private String nama_2002;
        private String nim_2002;
        private String prodi_2002;

        public Mahasiswa_2002(String nama_2002, String nim_2002, String prodi_2002) {
            this.nama_2002 = nama_2002;
            this.nim_2002 = nim_2002;
            this.prodi_2002 = prodi_2002;
        }

        public String getNama_2002() {
            return nama_2002;
        }

        public String getNim_2002() {
            return nim_2002;
        }

        public String getProdi_2002() {
            return prodi_2002;
        }
    }

```



```

@Override
public String toString() {
    return nama_2002;
}
}

ArrayList<Mahasiswa_2002> data_2002 = new ArrayList<>();

JTextField txtNama_2002;
JTextField txtNim_2002;
JTextField txtProdi_2002;

JComboBox<String> cmbSort_2002;

JTable tabel_2002;
DefaultTableModel model_2002;

JTextArea areaProses_2002;

JButton btnTambah_2002;
JButton btnHapus_2002;
JButton btnSort_2002;

public MahasiswaGUI_2511532002() {

    setTitle("Sorting Mahasiswa");
    setSize(800,600);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    JPanel panelInput_2002 = new JPanel(new GridLayout(5, 2));

    panelInput_2002.add(new JLabel("Nama"));
    txtNama_2002 = new JTextField();
    panelInput_2002.add(txtNama_2002);

    panelInput_2002.add(new JLabel("NIM"));
    txtNim_2002 = new JTextField();
    panelInput_2002.add(txtNim_2002);

    panelInput_2002.add(new JLabel("Program Studi"));
    txtProdi_2002 = new JTextField();
    panelInput_2002.add(txtProdi_2002);

    panelInput_2002.add(new JLabel("Pilih Sorting"));

    cmbSort_2002 = new JComboBox<>(new String[]{
        "Insertion Sort",
        "Selection Sort",
        "Bubble Sort"
    });
    panelInput_2002.add(cmbSort_2002);

    btnTambah_2002 = new JButton("Tambah");
    btnHapus_2002 = new JButton("Hapus");

    panelInput_2002.add(btnTambah_2002);
    panelInput_2002.add(btnHapus_2002);

    add(panelInput_2002, BorderLayout.NORTH);

    // ===== TABLE =====
    model_2002 = new DefaultTableModel(new String[]{"Nama", "NIM", "Prodi"}, 0);
    tabel_2002 = new JTable(model_2002);

    add(new JScrollPane(tabel_2002), BorderLayout.CENTER);

    // ===== BOTTOM PANEL =====
    JPanel panelBawah_2002 = new JPanel(new BorderLayout());

    btnSort_2002 = new JButton("Mulai Sorting");
    panelBawah_2002.add(btnSort_2002, BorderLayout.NORTH);

    areaProses_2002 = new JTextArea(10, 50);
    areaProses_2002.setEditable(false);

    panelBawah_2002.add(new JScrollPane(areaProses_2002), BorderLayout.CENTER);

    add(panelBawah_2002, BorderLayout.SOUTH);

    aksiButton_2002();
}

void aksiButton_2002() {

    // ===== TAMBAH =====
    btnTambah_2002.addActionListener(e -> {

        Mahasiswa_2002 mhs = new Mahasiswa_2002(
            txtNama_2002.getText(),
            txtNim_2002.getText(),
            txtProdi_2002.getText()
        );
    });
}

```

```

data_2002.add(mhs);

model_2002.addRow(new Object[]{
    mhs.getNama_2002(),
    mhs.getNim_2002(),
    mhs.getProdi_2002()
});

txtNama_2002.setText("");
txtNim_2002.setText("");
txtProdi_2002.setText("");
});

// ===== HAPUS (FIXED) =====
btnHapus_2002.addActionListener(e -> {

    int baris = tabel_2002.getSelectedRow();

    if (baris == -1) {
        JOptionPane.showMessageDialog(this, "Pilih data dulu!");
        return;
    }

    String nim = model_2002.getValueAt(baris, 1).toString();

    data_2002.removeIf(m -> m.getNim_2002().equals(nim));
    model_2002.removeRow(baris);
});

// ===== SORT =====
btnSort_2002.addActionListener(e -> {

    ArrayList<Mahasiswa_2002> temp = new ArrayList<>(data_2002);
    String pilih = cmbSort_2002.getSelectedItem().toString();

    if (pilih.equals("Insertion Sort")) {
        insertionSort_2002(temp);
    } else if (pilih.equals("Selection Sort")) {
        selectionSort_2002(temp);
    } else {
        bubbleSort_2002(temp);
    }
});

// ===== INSERTION SORT =====
void insertionSort_2002(ArrayList<Mahasiswa_2002> list) {

    areaProses_2002.append("\n=== INSERTION SORT ===\n");

    for (int i = 1; i < list.size(); i++) {
        Mahasiswa_2002 key = list.get(i);
        int j = i - 1;

        while (j >= 0 &&
            list.get(j).getNama_2002()
                .compareToIgnoreCase(key.getNama_2002()) > 0) {

            list.set(j + 1, list.get(j));
            j--;
        }

        list.set(j + 1, key);

        areaProses_2002.append("Langkah " + i + " : " + ambilNama_2002(list) + "\n");
    }
}

// ===== SELECTION SORT =====
void selectionSort_2002(ArrayList<Mahasiswa_2002> list) {

    areaProses_2002.append("\n=== SELECTION SORT ===\n");

    for (int i = 0; i < list.size() - 1; i++) {

        int min = i;

        for (int j = i + 1; j < list.size(); j++) {

            if (list.get(j).getNama_2002()
                .compareToIgnoreCase(list.get(min).getNama_2002()) < 0) {
                min = j;
            }
        }

        Mahasiswa_2002 temp = list.get(i);
        list.set(i, list.get(min));
        list.set(min, temp);

        areaProses_2002.append("Pass " + (i + 1) + " : " + ambilNama_2002(list) + "\n");
    }
}

// ===== BUBBLE SORT =====

```

```

void bubbleSort_2002(ArrayList<Mahasiswa_2002> list) {
    areaProses_2002.append("\n=== BUBBLE SORT ===\n");
    for (int i = 0; i < list.size() - 1; i++) {
        for (int j = 0; j < list.size() - i - 1; j++) {
            if (list.get(j).getNama_2002()
                .compareToIgnoreCase(list.get(j + 1).getNama_2002()) > 0) {
                Mahasiswa_2002 temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
            }
        }
        areaProses_2002.append("Pass " + (i + 1) + " : " + ambilNama_2002(list) + "\n");
    }
}

String ambilNama_2002(ArrayList<Mahasiswa_2002> list) {
    String hasil = "[ ";
    for (Mahasiswa_2002 m : list) {
        hasil += m.getNama_2002() + " ";
    }
    return hasil + "]";
}

public static void main(String[] args) {
    new MahasiswaGUI_2511532002().setVisible(true);
}
}

```

Penjelasan:

1) Insertion Sort

// ===== INSERTION SORT =====

```

void insertionSort_2002(ArrayList<Mahasiswa_2002> list) {
    areaProses_2002.append("\n=== INSERTION SORT ===\n");
    for (int i = 1; i < list.size(); i++) {

        Mahasiswa_2002 key = list.get(i);
        int j = i - 1;

        while (j >= 0 &&
            list.get(j).getNama_2002()
                .compareToIgnoreCase(key.getNama_2002()) > 0) {
            list.set(j + 1, list.get(j));
            j--;
        }

        list.set(j + 1, key);
        areaProses_2002.append("Langkah " + i + " : " +
            ambilNama_2002(list) + "\n");
    }
}

```

Insertion Sort digunakan untuk mengurutkan data mahasiswa berdasarkan nama secara *ascending* (A-Z) dengan cara menyisipkan elemen pada posisi yang sesuai.

2) Selection Sort

```
// ===== SELECTION SORT =====
void selectionSort_2002(ArrayList<Mahasiswa_2002> list) {
    areaProses_2002.append("\n=== SELECTION SORT ===\n");

    for (int i = 0; i < list.size() - 1; i++) {
        int min = i;
        for (int j = i + 1; j < list.size(); j++) {

            if (list.get(j).getNama_2002()
                .compareToIgnoreCase(list.get(min).getNama_2002()) < 0)
            {
                min = j;
            }
        }
        Mahasiswa_2002 temp = list.get(i);
        list.set(i, list.get(min));
        list.set(min, temp);

        areaProses_2002.append("Pass " + (i + 1) + " : " +
            ambilNama_2002(list) + "\n");
    }
}
```

Selection Sort digunakan untuk mencari data dengan nilai terkecil (nama A-Z) kemudian menukarnya ke posisi awal secara berulang sampai data terurut.

3) Bubble Sort

```
// ===== BUBBLE SORT =====
void bubbleSort_2002(ArrayList<Mahasiswa_2002> list) {
    areaProses_2002.append("\n=== BUBBLE SORT ===\n");
    for (int i = 0; i < list.size() - 1; i++) {
        for (int j = 0; j < list.size() - i - 1; j++) {
            if (list.get(j).getNama_2002()
                .compareToIgnoreCase(list.get(j + 1).getNama_2002()) > 0) {

                Mahasiswa_2002 temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
            }
        }

        areaProses_2002.append("Pass " + (i + 1) + " : " +
            ambilNama_2002(list) + "\n");
    }
}

String ambilNama_2002(ArrayList<Mahasiswa_2002> list) {
```

```

String hasil = "[ ";
for (Mahasiswa_2002 m : list) {
    hasil += m.getNama_2002() + " ";
}
return hasil + "]";
}

```

Bubble Sort digunakan untuk membandingkan data yang berdekatan dan menukarnya jika urutan salah, dilakukan berulang hingga data terurut.

4) *Method Main*

```

public static void main(String[] args) {
    new MahasiswaGUI_2511532002().setVisible(true);
}
}

```

Method utama yang digunakan untuk menjalankan program dan menampilkan *GUI* aplikasi.

2. Hasil

1) *Insertion Sort*

Tambah			Hapus		
Nama	NIM	Prodi			
Gina	2511533014	Rekayasa Mesin			
Annisa	2511532024	Arsitektur			
Sasa	2511531018	Rekayasa Elektro			
Dinda	2511531020	Hubungan Internasional			
Naya	2511531016	Ilmu Hitam			

==== INSERTION SORT ====

Langkah 1 : [Annisa Gina Sasa Dinda Naya]

Langkah 2 : [Annisa Gina Sasa Dinda Naya]

Langkah 3 : [Annisa Dinda Gina Sasa Naya]

Langkah 4 : [Annisa Dinda Gina Naya Sasa]

==== SELECTION SORT ====

Pass 1 : [Annisa Gina Sasa Dinda Naya]

Pass 2 : [Annisa Dinda Sasa Gina Naya]

2) Selection Sort

Sorting Mahasiswa

Nama

NIM

Program Studi

Pilih Sorting **Bubble Sort**

Tambah		Hapus
Nama	NIM	Prodi
Gina	2511533014	Rekayasa Mesin
Annisa	2511532024	Arsitektur
Sasa	2511531018	Rekayasa Elektro
Dinda	2511531020	Hubungan Internasional
Naya	2511531016	Ilmu Hitam

Mulai Sorting

```

=== SELECTION SORT ===
Pass 1 : [ Annisa Gina Sasa Dinda Naya ]
Pass 2 : [ Annisa Dinda Sasa Gina Naya ]
Pass 3 : [ Annisa Dinda Gina Sasa Naya ]
Pass 4 : [ Annisa Dinda Gina Naya Sasa ]

=== BUBBLE SORT ===
Pass 1 : [ Annisa Gina Dinda Naya Sasa ]
Pass 2 : [ Annisa Dinda Gina Naya Sasa ]

```

3) Bubble Sort

Sorting Mahasiswa

Nama

NIM

Program Studi

Pilih Sorting **Bubble Sort**

Tambah		Hapus
Nama	NIM	Prodi
Gina	2511533014	Rekayasa Mesin
Annisa	2511532024	Arsitektur
Sasa	2511531018	Rekayasa Elektro
Dinda	2511531020	Hubungan Internasional
Naya	2511531016	Ilmu Hitam

Mulai Sorting

```

Pass 2 : [ Annisa Dinda Sasa Gina Naya ]
Pass 3 : [ Annisa Dinda Gina Sasa Naya ]
Pass 4 : [ Annisa Dinda Gina Naya Sasa ]
]

=== BUBBLE SORT ===
Pass 1 : [ Annisa Gina Dinda Naya Sasa ]
Pass 2 : [ Annisa Dinda Gina Naya Sasa ]
Pass 3 : [ Annisa Dinda Gina Naya Sasa ]
Pass 4 : [ Annisa Dinda Gina Naya Sasa ]

```

3. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* berhasil diimplementasikan menggunakan bahasa pemrograman Java. Program juga berhasil menampilkan proses pengurutan data mahasiswa secara alfabetis menggunakan *Java GUI (Swing)*.

Melalui praktikum ini, penulis dapat memahami cara kerja masing-masing algoritma *sorting*, penggunaan *compareToIgnoreCase()* untuk membandingkan *string*, serta visualisasi proses *sorting* langkah demi langkah pada *GUI Java*.